

ENGR1050: Lecture 12

Exam recap, intro to NumPy, solving linear systems

October 6, 2025



Agenda

- Go over Exam 1 grades and solution
- Some linear algebra review
- Introduction to NumPy

Exam 1

Some reminders:

- Exam 1 was to assess your grasp of fundamentals before we move into scientific computing
- Overall, the class did very well.
- If you performed less well than you hoped, remember a few things:
 - We will continue to use these fundamentals in the course, so you will have more practice
 - We drop the lowest exam score to make it easier to catch up
 - Come to office hours!!!! Many have started coming and made great progress

New TAs: We now have two undergraduate TAs who will give you even more chances to get help. They will be in the GM lab 11:30am-3:30pm on Tuesdays. Their names are Kayleen Smith and Annaliese Niebauer and they'll leave a placard on the desk that says **ENGR1050**.

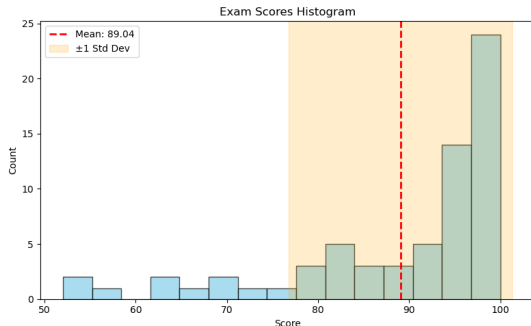
Exam 1 Grades

Statistics:

- Average: 89
- Median: 95
- Standard Deviation: 12.3
- Highest Score: 100
- Lowest Score: 52

Solutions:

[Open in Colab](#)



Onward to scientific computing!

- We will now start learning how to use Python for scientific computing
- Today, we will introduce NumPy, the fundamental package for scientific computing in Python
- NumPy provides support for arrays (vectors/matrices) and many functions to manipulate them
- It is also the basis for many other scientific computing packages in Python, including SciPy, pandas, and matplotlib

This unit of the course will be more mathematical - don't get caught off guard! Come to OH early!

Scalars, vectors, and matrices

- A **scalar** is a single number, e.g. $x = 5$
- A **vector** is an ordered list of numbers, e.g. $\mathbf{v} = [1, 2, 3]$
- A **matrix** is a 2D array of numbers, e.g.

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

- Vectors and matrices are fundamental in scientific computing for representing data and performing calculations

Rules to manipulate scalars

- Addition: $a + b$
- Subtraction: $a - b$
- Multiplication: $a \times b$ or $a \cdot b$
- Division: $\frac{a}{b}$ or a/b
- Exponentiation: a^b or $a ** b$
- Order of operations follows standard mathematical rules (PEMDAS/BODMAS)

Rules to manipulate vectors

Think of vectors as a list of numbers,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

with some additional rules:

- Addition: $\mathbf{u} + \mathbf{v} = [u_1 + v_1, u_2 + v_2, \dots]$
- Subtraction: $\mathbf{u} - \mathbf{v} = [u_1 - v_1, u_2 - v_2, \dots]$
- Scalar multiplication: $c \cdot \mathbf{v} = [c \cdot v_1, c \cdot v_2, \dots]$
- Dot product: $\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots$
- Magnitude (length): $\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots}$

Rules to manipulate matrices

Think of matrices as a 2D array of numbers:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

with some additional rules:

- Addition: $\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}]$ (element-wise)
- Subtraction: $\mathbf{A} - \mathbf{B} = [a_{ij} - b_{ij}]$ (element-wise)
- Scalar multiplication: $c \cdot \mathbf{A} = [c \cdot a_{ij}]$
- Matrix multiplication: $\mathbf{AB}$ (dot product of rows and columns)
- Transpose: $\mathbf{A}^T$ (swap rows and columns: $a_{ij} \rightarrow a_{ji}$)

For some this will be your first encounter with matrices - we will spend time on them in a future lecture.

NumPy scalars/vectors/matrices

- NumPy provides support for scalars, vectors, and matrices through its *numpy array* object
- You can create a NumPy array using the `np.array()` function

```
import numpy as np

# Create a 0D array (scalar)
s = np.array(5)

# Create a 1D array (vector)
v = np.array([1, 2, 3])

# Create a 2D array (matrix)
M = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Element-wise operations
v2 = np.array([4, 5, 6])
v3 = v + 2.3 * v2
# v3 = [1+4*2.3,
#       2+5*2.3,
#       3+6*2.3]
```

np.array syntax

```
import numpy as np
```

```
# From Python lists
```

```
a = np.array([1, 2, 3])           # 1D array
```

```
b = np.array([[1, 2], [3, 4]])    # 2D array
```

```
# Special arrays
```

```
np.zeros((2, 3))                  # 2x3 array of zeros
```

```
np.ones((3,))                     # 1D array of ones
```

```
np.eye(3)                         # 3x3 identity matrix
```

```
np.arange(0, 10, 2)               # [0, 2, 4, 6, 8]
```

```
np.linspace(0, 1, 5)              # [0., 0.25, 0.5, 0.75, 1.]
```

Two ways to construct:

- From a list for a vector, or list of lists for 2D arrays
- Using special functions like `np.zeros()`, `np.ones()`, `np.eye()`, `np.arange()`, and `np.linspace()`

Accessing np.array properties and elements

```
a = np.array([1, 2, 3])           # 1D array
b = np.array([[1, 2], [3, 4]])    # 2D array
```

Properties of np.arrays

```
a.shape      # (3,)           → dimensions
a.ndim       # 1              → number of axes
a.size       # 3              → total number of elements
a.dtype      # dtype('int64') → data type
```

Indexing/slicing

```
a[0]         # first element
b[1, 0]      # row 1, column 0
a[1:]        # slice from index 1 to end
b[:, 1]      # all rows, column 1
b[0, :]      # first row
```

- Functions to get shapes of arrays
- Can index/slice like Python lists, but also with multiple dimensions

Scalar Multiplication

We've got a pretty good grip on scalar multiplication:

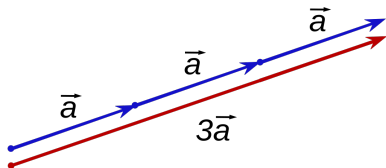
```
import numpy as np
A = np.array([2])
B = np.array([3])
C = A * B # or np.multiply(A, B)
print(C) # [6]
```

Vector Multiplication

Comes in two flavors:

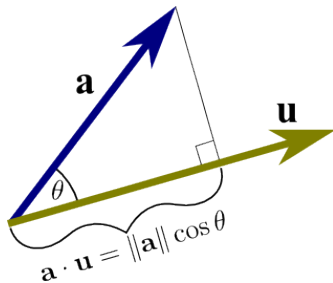
Scalar multiplication

$$\alpha \mathbf{v} = [\alpha v_1, \alpha v_2, \dots]$$



Dot multiplication

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots$$



Vector Multiplication in code

Comes in two flavors:

Scalar multiplication

$$\alpha \mathbf{v} = [\alpha v_1, \alpha v_2, \dots]$$

```
import numpy as np
alpha = np.array([2])
v = np.array([3, 4, 5])
out = alpha * v # or np.multiply(alpha, v)
print(out) # [6, 8, 10]
```

Dot multiplication

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots$$

```
import numpy as np
u = np.array([2, 3, 4])
v = np.array([5, 6, 7])
out = np.dot(u, v) # or u @ v
print(out) # 56
```

Matrix Multiplication

- Matrix multiplication combines rows of the first matrix with columns of the second.
- For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the product $C = AB$ has entries

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

- Dimensions must satisfy: (*inner dimensions match*) $n = n$.

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[2, 0],
              [1, 3]])
C = A @ B      # or np.dot(A, B)
print(C)
# [[ 4  6]
#  [10 12]]
```

Example: Writing a System as a Matrix Product

Start with two equations

$$2x + y = 5,$$

$$x - y = 1.$$

- Collect the coefficients of x and y into a **matrix** A :

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}.$$

- Multiply $A\mathbf{x}$ row-by-row:

$$A\mathbf{x} = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2x + 1y \\ 1x - 1y \end{bmatrix}.$$

- Setting this equal to $\mathbf{b}$ gives back the original system:

$$\begin{cases} 2x + y = 5, \\ x - y = 1. \end{cases}$$

Linear Systems as Matrix Equations

- A system of linear equations can be written compactly as

$$A\mathbf{x} = \mathbf{b},$$

where

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

- Each equation corresponds to **one row** of A :

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1,$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2,$$

$$\vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n.$$

Today's exercise: Gaussian elimination

- Goal: Solve a system of linear equations $A\mathbf{x} = \mathbf{b}$
- Operations allowed (think back to your high school algebra class):
 - 1 Swap two rows.
 - 2 Multiply a row by a nonzero scalar.
 - 3 Add a multiple of one row to another.

Example: A 3×3 Linear System

Solve

$$\begin{aligned}x + y + z &= 6, \\2x - y + z &= 3, \\3x + y + 2z &= 11.\end{aligned}$$

Step 1. Write as a matrix equation

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & -1 & 1 \\ 3 & 1 & 2 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 6 \\ 3 \\ 11 \end{bmatrix}.$$

Example: A 3×3 Linear System

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & -1 & 1 \\ 3 & 1 & 2 \end{bmatrix} x = \begin{bmatrix} 6 \\ 3 \\ 11 \end{bmatrix}.$$

Step 2. Eliminate x from rows 2 and 3 using row 1.

$$R_2 \leftarrow R_2 - 2R_1, \quad R_3 \leftarrow R_3 - 3R_1$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -3 & -1 \\ 0 & -2 & -1 \end{bmatrix} x = \begin{bmatrix} 6 \\ -9 \\ -7 \end{bmatrix}.$$

Example: A 3×3 Linear System

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -3 & -1 \\ 0 & -2 & -1 \end{bmatrix} x = \begin{bmatrix} 6 \\ -9 \\ -7 \end{bmatrix}.$$

Step 3. Eliminate y from row 3 using an integer combination to avoid fractions.

$$R_3 \leftarrow 3R_3 - 2R_2$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -3 & -1 \\ 0 & 0 & -1 \end{bmatrix} x = \begin{bmatrix} 6 \\ -9 \\ -3 \end{bmatrix}.$$

Example: A 3×3 Linear System

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -3 & -1 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ -9 \\ -3 \end{bmatrix}.$$

Step 4. "Backsolve" by substituting from the bottom up:

- 1 From row 3: $-x_3 = -3 \Rightarrow x_3 = 3$.
- 2 From row 2: $-3x_2 - x_3 = -9 \Rightarrow -3x_2 - 3 = -9 \Rightarrow x_2 = 2$.
- 3 From row 1: $x_1 + x_2 + x_3 = 6 \Rightarrow x_1 + 2 + 3 = 6 \Rightarrow x_1 = 1$.

Today's exercise

Repeat the last 4 steps in Python using NumPy.

[Click here to open the Jupyter Notebook for Lecture 12](#)